

claude-windows / b13673cc-f087-43b1-9527-35dc0b67b301

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\b13673cc-f087-43b1-9527-35dc0b67b301\subagents\workflows\wf_61401384-db8\agent-aa87e3f64fe8da19a.jsonl`  
- SHA-256: `09700e0f2d03298dc8808628a1ac48b09d63b381785dbe1b75a49a89830f247d`  
- Source modified: `2026-06-06T06:47:33+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_61401384-db8`  
- session_id: `b13673cc-f087-43b1-9527-35dc0b67b301`
```

Transcript

1. user (2026-06-06T06:45:13.282Z)

You are checking what the user actually ASKED Roora to deliver, so deviations can be characterized fairly and WITHOUT fabrication. Ground every claim in the actual documents ? quote them. If a document is silent on something, say "spec is silent", do not invent.

KEY PATHS (all local, absolute, read with the Read/Bash/Grep tools):

```
- Roora delivery root: "C:\Users\avidu\Projects\Annotation Setup\6 June 2026"  
  - "badminton\" = PASCAL VOC export (Annotations\frame_XXXXXX.xml per sampled frame, ImageSets\, labelmap.txt)  
    - "badminton (1)\annotations\instances_Test.json" = COCO export  
    - "badminton (2)\annotations.xml" = CVAT-for-video native export (has <meta><job> with frame_filter step, stop_frame, labels)  
- Roora spec PDFs the user sent the vendor:  
  - "C:\Users\avidu\Projects\Annotation Setup\Roora Annotation Brief ? Rally Golden Set (Format Spec, Scope, QA).pdf"  
  - "C:\Users\avidu\Projects\Annotation Setup\Illustrated Annotation Guide ? Rally Labeling (Supplement).pdf"  
- The three sample videos given: "G:\My Drive\Sharing\Annotation Videos\" (Video 1 - Badminton.MP4, Video 2 - Pickleball.mp4, Video 3 - Tabletennis.mp4)  
- Indexer repo: C:\Users\avidu\Projects\badminton-highlight-indexer  
- Collector repo: C:\Users\avidu\Projects\sports-data-collector  
- python available as 'python' (3.13); ffmpeg available.
```

FACTS ALREADY ESTABLISHED (verify, don't just trust; correct me if any is wrong):

```
- ffmpeg on the on-disk videos: Badminton = 5312x2988, 29.97fps, 100.7s, 3019 frames.  
Pickleball = 3840x2160, 59.94fps, ~97s. Tabletennis = 3840x2160, 59.94fps, ~58s.  
- CVAT job meta (badminton (2)/annotations.xml): stop_frame=21270, frame_filter step=30, size=710 sampled frames, annotation canvas 1920x1080, mode=interpolation, label="rally".  
- All three folders appear to be the SAME badminton job exported in 3 formats (COCO bbox [1134.1,434.73,152.19,270.73] == VOC rally-2 box xmin1134.1/ymin434.73/xmax1286.29/ymax705.46). Pickleball & table-tennis are NOT in the delivery.  
- The rally fields delivered (as CVAT object attributes, attached to a per-player bounding box): rally_number, start_time, end_time, shots_count, ending_reason, score_before (A-B), score_after (A-B), notes, rotation.  
- VOC has 24 distinct rally_numbers; 281/710 frames contain >=1 rally object; COCO has 318 annotations over 710 images.  
- HYPOTHESIS to verify: start_time/end_time attributes == source_frame/900 (e.g. rally 24 frame 19920->20160 gives 22.13->22.4; 19920/900=22.13). i.e. attribute times are off by the sampling-step factor and the TRUE temporal signal is the per-object FRAME RANGE, not the start_time/end_time attributes.  
- HYPOTHESIS: the annotated badminton source (21270 frames, 1080p) is NOT the on-disk Video 1 - Badminton.MP4 (3019 frames, 5312x2988).  
- ending_reason values used: forced_error, unforced_error, service_fault, let, other, winner.
```

TASK:

- Read both spec PDFs (use Read with the pages param; they are small). Also read the collector source-of-truth docs: docs/annotation/ROORA_BRIEF.md, docs/annotation/ANNOTATION_GUIDE.md, docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv in C:\Users\avidu\Projects\sports-data-collector.
- Establish: what OUTPUT FORMAT was requested (a CSV of per-rally rows? bounding boxes? CVAT?), the exact requested fields, the exact requested ending_reason vocabulary, and the requested pilot scope (how many videos / which sports).
- Then list deviations between requested and what Roora delivered (per KNOWN_FACTS): format (CSV vs CVAT bbox export), ending_reason vocab, the broken time computation, scope (only badminton delivered), any extra/missing fields.

Quote the doc text as evidence for each "requested" claim.

2. assistant (2026-06-06T06:45:16.335Z)

I'll start by reading the spec PDFs and the collector source-of-truth docs to establish what was actually requested.

3. user (2026-06-06T06:45:17.313Z)

PDF file read: C:\Users\avidu\Projects\Annotation Setup\Roora Annotation Brief ? Rally Golden Set (Format Spec, Scope, QA).pdf (79KB)

4. user (2026-06-06T06:45:17.849Z)

PDF file read: C:\Users\avidu\Projects\Annotation Setup\Illustrated Annotation Guide ? Rally Labeling (Supplement).pdf (111.8KB)

5. user (2026-06-06T06:45:18.759Z)

```
1      # Roora Annotation Brief ? Rally Golden Set
2
3      **Project:** racket-sport rally segmentation golden / regression dataset
4      **Sports in scope:** Badminton, Pickleball, Table Tennis (each in **singles_and_
doubles**)
5      **Companion doc:** [ `ANNOTATION_GUIDE.md` ] (./ANNOTATION_GUIDE.md)
6
7      > This brief is the vendor-facing spec. The annotation deliverable is a per-rally CSV
that
8      > imports directly via `python -m src.cli.curate import-local`.
9
10     ## 0. What we need in one sentence
11     For each provided match video, produce **one CSV file** listing **every rally** with its
12     start time, end time, number of shots, and how it ended ? **excluding all dead time**.
13
14     ## 1. Deliverable format (canonical)
15     One CSV per video (UTF-8), named exactly `.csv` (we supply the `video_id`).
16     One row per rally, chronological, header row required.
17
18     | column | meaning |
19     |---|---|
20     | `rally_number` | integer, 1-based, chronological. No gaps, no repeats. |
21     | `start_mmss` | rater-entered time as `m:ss.mmm` (e.g. `1:12.40`) ? serve contact. |
22     | `end_mmss` | rater-entered time as `m:ss.mmm` ? ball/shuttle dead. |
23     | `start_time` | decimal **seconds** (e.g. `72.40`). **Auto-filled** from `start_mmss`.
|
24     | `end_time` | decimal **seconds**. **Auto-filled** from `end_mmss`. |
25     | `shots_count` | integer. Total racket/paddle/bat contacts (serve = shot 1). |
26     | `ending_reason` | one of: `winner` \ | `into_net` \ | `out` \ | `service_fault` \ | `let`
\ | `other` |
27     | `score_before` | optional, `"A-B"`. Blank if not confidently readable. |
28     | `score_after` | optional, `"A-B"`. |
29     | `notes` | optional free text. **Required** when `ending_reason = other`. |
30
31     **Times (mm:ss ? decimal seconds).** Raters type only `start_mmss`/`end_mmss`. The
```

```

template
32     Google Sheet fills `start_time`/`end_time` with this formula (shown for `start_time`,
row 2):
33
34     ```
35     =IF(B2="", "", IFERROR(VALUE(LEFT(B2,FIND(":",B2)-1))*60 +
VALUE(MID(B2,FIND(":",B2)+1,20)), VALUE(B2)))
36     ```
37
38     On Download as CSV, the computed decimal seconds are exported. (If your tool already
39     shows decimal seconds, type those into `start_time`/`end_time` and leave the mm:ss
columns blank.)
40
41     Example (badminton):
42     ```csv
43     rally_number,start_mmss,end_mmss,start_time,end_time,shots_count,ending_reason,score_bef
ore,score_after,notes
44     1,0:12.50,0:25.00,12.50,25.00,9,winner,0-0,1-0,
45     2,0:45.00,0:47.30,45.00,47.30,1,service_fault,1-0,1-1,serve into net
46     3,1:01.20,1:18.85,61.20,78.85,14,out,1-1,2-1,final clear landed long
47     ```
48
49     Critical format rules
50     - `start_time`/`end_time` that reach us must be decimal seconds (the formula
guarantees this).
51     - Times are relative to the start of the file we provide. Do not trim, clip, or
52     re-encode the video ? our pipeline identifies a video by its file checksum.
53     - `end_time` > `start_time` for every row.
54     - No overlaps: `end_time[N]` <= `start_time[N+1]`.
55     - Everything between one rally's `end_time` and the next rally's `start_time` is dead
time and gets no row.
56
57     ## 2. The three non-negotiable rules (full detail + pictures in the Guide)
58     1. Ignore all dead time. Only label active play.
59     2. Include every rally ? exclude none. Every served point, including service
faults,
60     1?2 shot rallies, and lets/replays. A missed rally is the worst error in this
project.
61     Exception: warm-up / knock-up is NOT a rally ? pre-match and between-games warm-
up looks
62     like rallying but isn't scored play, so don't label it. Tell it apart by: no real
serve,
63     cooperative (not competitive) hitting, the score not progressing, often several
shuttles/balls
64     in use (full how-to in the Guide, Part 2). When unsure if the match has started, flag
it.
65     3. A shot = one contact with the racket/paddle/bat. Serve = shot 1. Bounces and net-
clips are not shots.
66
67     ## 3. Scope & phasing
68     Phase A ? Pilot (this engagement): 3 videos (1 badminton + 1 pickleball + 1 table
tennis),
69     each < 3 min with ~10?12 rallies. This is the paid calibration round; we review
jointly and
70     lock the Guide before scale-up.
71
72     Phase B ? Scale (after a successful pilot): ~12?15 videos per sport (~36?45
total), each
73     ~30 min. Same format, rules, and template.
74
75     > Doubles are included across the program. Rally rules are identical to singles; the
76     > badminton- and pickleball-specific doubles notes are in the Guide.
77
78     ## 4. Quality bar (acceptance criteria)
79     Automated validation (no overlaps, positive durations, schema-valid) runs on every file,

```

```

then a
80     ~20% human spot-check. A file is returned for rework (in scope) if it misses:
81     - Completeness: every rally present; zero missed rallies; zero dead-time rows.
82     - Boundaries: `start_time`/`end_time` within  $\pm 0.3$  s of true serve-contact /
landing.
83     - shots_count: exact for rallies ? 15 shots;  $\pm 1$  allowed for longer/faster rallies.
84     - ending_reason: one of the six allowed values, per the Guide's decision tree.
85     - Format: decimal seconds present, valid columns, non-overlapping, chronological.
86
87     ## 5. Per-sport quick reference (full version + examples in the Guide)
88     Badminton (singles & doubles) ? start: racket?shuttle serve contact . end: shuttle
floors / fault . shot: each racket?shuttle contact.
89     Pickleball (singles & doubles) ? start: paddle?ball serve contact . end: double
bounce / out / net / fault . shot: each paddle?ball contact (floor bounces are not shots;
two-bounce rule is normal play).
90     Table tennis ? start: racket?ball serve strike (not the toss) . end: failed legal
return . shot: each racket?ball contact (table bounces are not shots). Net-cord serve =
`let`.
91
92     ## 6. ending_reason values (observational ? decision tree in the Guide)
93     | value | meaning |
94     |---|---|
95     | `winner` | last shot landed in and the opponent made no legal contact. |
96     | `into_net` | the final (non-serve) shot went into the net. |
97     | `out` | the final (non-serve) shot landed outside the court/table. |
98     | `service_fault` | the serve itself failed / was illegal. |
99     | `let` | the point was replayed. |
100    | `other` | anything else / cannot tell. Must add a note. |
101
102    ## 7. Recommended capture method
103    Use any tool that shows current time (mm:ss.mmm or seconds, or frame + known fps) and
can
104    step one frame forward/back: a frame-steppable player (VLC frame-step) or a timeline
tool
105    (Label Studio / CVAT). The deliverable is the CSV in the schema above.
106    `seconds = frame_number / fps` (we provide fps per video).
107
108    ## 8. Logistics & contacts
109    - TODO(avidullu): paste the Google Drive folder link** where source videos are shared
with Roora.
110    - Per-video `fps` + `video_id`: provided in the intake manifest ? template
[`INTAKE_MANIFEST_TEMPLATE.csv`](./INTAKE_MANIFEST_TEMPLATE.csv). TODO(avidullu): fill the
manifest (one row per pilot video) and share its link.**
111    - TODO(avidullu): paste the Drive folder link** where Roora uploads completed
`<video_id>.csv` files.
112    - Blank annotation template Sheet (mm:ss ? seconds formula from $1 pre-filled):
TODO(avidullu): create it and paste the link.**
113    - TODO(avidullu): add the escalation channel** (email / whatsapp group) for edge-case
questions.
114    - NDA before any footage is shared ? template [`NDA_TEMPLATE.md`](./NDA_TEMPLATE.md).
TODO(avidullu): have counsel review, fill the blanks, and have Roora sign.**
115    - Roora contact on file: `info@rooragr.com` (HSR Layout, Bangalore).
116
117    ## 9. Pricing request (please quote)
118    1. Price per rally and per minute of footage.
119    2. Pilot price (3 short videos, ~30?36 rallies) as a paid calibration round.
120    3. Indicative scale price for ~36?45 longer videos.
121    4. Turnaround for the pilot and throughput (rallies/day) at scale.
122    5. QA process and rework policy.
123    6. Whether annotators have racket-sport familiarity (helpful, not required).
124    7. Confirmation you can deliver the CSV schema above.
125

```

```

1      # Illustrated Annotation Guide ? Rally Labeling
2
3      Supplement to [R00RA_BRIEF.md](./R00RA_BRIEF.md). Sports: **Badminton, Pickleball,
Table Tennis** (singles **and** doubles).
4
5      > **Illustrations:** diagrams are drawn in text so they survive copy/paste. Boxes marked
6      > `[SCREENSHOT: ...]` are placeholders.
7      > **TODO(avidullu):** replace every `[SCREENSHOT: ...]` placeholder with a real frame grab
8      > from the pilot videos before sending this guide to Roora.**
9
10     ## Part 1 ? What you are producing
11     One CSV per video (from the template Sheet), one row per rally:
12
13     ```
14     rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason,
score_before, score_after, notes
15     ```
16
17     You type: `rally_number`, `start_mmss`, `end_mmss`, `shots_count`, `ending_reason` (+
optional
18     score/notes). The `start_time`/`end_time` decimal-second columns **fill themselves** via
a
19     formula already in the template.
20
21     For every rally you record:
22     - **When it started** (serve contact) ? `start_mmss` (`m:ss.mmm`, e.g. `1:12.40`)
23     - **When it ended** (shuttle/ball dead) ? `end_mmss`
24     - **How many shots** happened ? `shots_count`
25     - **How it ended** ? `ending_reason`
26
27     Everything else (gaps between rallies) you **ignore**.
28
29     ## Part 2 ? The three golden rules
30
31     ### Rule 1: Ignore all dead time
32     A rally is **continuous active play**. The moment play stops, the rally is over. The
time
33     until the next serve is **dead time** and is never labeled.
34
35     ```
36     DEAD          RALLY 1          DEAD          RALLY 2          DEAD
37     (ignore) |=====| (ignore) |=====| (ignore)
38           ^               ^               ^               ^
39           serve          shuttle        serve          shuttle
40           contact        lands          contact        lands
41           =start_1       =end_1         =start_2       =end_2
42     ```
43
44     Dead time includes: walking back to serve, picking up the shuttle/ball, wiping sweat,
45     bouncing the ball before serving, the score being announced, replays/slow-mo, crowd or
coach
46     shots, changeovers, towel/medical timeouts, and **pre-match / between-games warm-up
47     (knock-up)** ? see the Rule 2 caveat below.
48
49     ### Rule 2: Include every rally ? even the ugly ones
50     Every served point gets a row, including:
51     - **Service faults** (serve into net/out/illegal) ? usually `shots_count = 1`,
`ending_reason = service_fault`.
52     - **Very short rallies** (serve + one error).
53     - **Lets / replays** ? `ending_reason = let`; the replayed point is its **own** next
row.
54
55     A **missed rally is the worst error** in this project ? the model must learn that short
and
56     failed rallies are still rallies.

```

```

57
58 > **?? BUT warm-up / knock-up is NOT a rally** ? a common trap, because Rule 2 says
"include
59 > everything." Pre-match and between-games warm-up *looks* like rallying but is not
scored play;
60 > **do not label it.** How to tell it apart:
61 > - **No real serve.** Real play starts with a proper serve from the service position
(diagonal,
62 > behind the service line). Warm-up has players feeding cooperatively from mid-court
with no
63 > serve ? not a rally.
64 > - **Cooperative, not competitive.** Gentle hits straight to each other; nobody is
trying to win
65 > the point (no smashes-to-win, no scrambling/diving).
66 > - **Score isn't progressing.** No umpire / scoreboard not running / players casually
retrieve
67 > and re-feed; the score doesn't change. Often **several shuttles/balls** in use at
once.
68 > - **Per sport:** badminton & pickleball ? real play starts with the underhand serve
from the
69 > service court / behind the baseline; table tennis ? real play starts with the
tossed,
70 > two-bounce serve (knock-up is cooperative rallying without it).
71 >
72 > **When unsure:** find the first proper serve where the score starts to progress ?
labeling
73 > begins there. If you genuinely can't tell whether the match has started, **flag the
clip to us
74 > ? don't guess.**
75
76 ### Rule 3: A shot = one contact with the racket / paddle / bat
77 Count every time the shuttle/ball touches a racket, paddle, or bat. The serve is shot 1.
78 - **Count:** any racket/paddle/bat contact (serve, return, smash, block?).
79 - **Don't count:** floor bounces, table bounces, net clips, the toss before a serve.
80
81 5-shot example: `serve(1) ? return(2) ? hit(3) ? hit(4) ? winner(5)` ? `shots_count =
5`, `ending_reason = winner`.
82
83 ## Part 3 ? Where exactly does a rally start and end?
84 **START (`start_mmss`):** the exact frame the server's racket/paddle/bat **contacts**
the
85 shuttle/ball. Not the toss, not the wind-up ? the contact frame.
86 `[SCREENSHOT: serve-contact frame]`
87
88 **END (`end_mmss`):** the exact frame the shuttle/ball becomes **dead**:
89 - Badminton: shuttle touches the floor, or play stops on a fault.
90 - Pickleball: the second floor bounce, or the frame the ball hits the net / lands out.
91 - Table tennis: the second bounce on one side, a non-serve net hit, or off the table.
92
93 `[SCREENSHOT: end frame]`
94
95 If occluded/off-screen, pick the closest clear frame and add a note.
96 **Boundary tolerance:  $\pm 0.3$  s** (~9 frames at 30 fps).
97
98 ## Part 4 ? ending_reason: how to choose (observational)
99 Pick exactly **one** value by looking at the **last shot** and what physically happened.
100 Stop at the first match:
101
102 1. Did the **serve** itself fail / was illegal (and ended the point)? ? `service_fault`
103 2. Was the point **replayed** (let / interruption)? ? `let`
104 3. Did the final (non-serve) shot go **into the net**? ? `into_net`
105 4. Did the final (non-serve) shot land **out**? ? `out`
106 5. Did the final shot land **in** and the opponent make **no legal return**? ? `winner`
107 6. None of the above / cannot tell ? `other` (add a note)
108

```

```

109 Notes:
110 - Attribute the reason to the final contact's outcome. If a player reached the ball
but
111 their return netted or went out, that is `into_net`/`out` (their shot), not `winner`.
112 - A serve into the net is `service_fault` (rule 1 beats `into_net`).
113
114 ## Part 5 ? Per-sport detail
115
116 ### Badminton
117 - Serve: underhand, below the waist, diagonal. Start = racket?shuttle contact.
118 - Rally end: shuttle hits the floor, or play stops on a fault.
119 - Shots: every racket?shuttle contact; serve = shot 1.
120 - Service faults (1-shot rally, `service_fault`): serve into net, serve outside
service court, contact above waist, foot fault.
121
122 Worked example:
123 ```
124 [SCREENSHOT: badminton serve]
125 Rally 4: serve 1:28.30, long rally, smash winner, shuttle lands 1:44.10, 12 contacts.
126 4, 1:28.30, 1:44.10, (auto), (auto), 12, winner, 3-5, 4-5,
127 Rally 5: serve 1:57.00 straight into the net.
128 5, 1:57.00, 1:57.90, (auto), (auto), 1, service_fault, 4-5, 4-6,
129 ```
130
131 Badminton ? DOUBLES (2 players per side). All rally rules are identical to
singles:
132 - `start` = serve contact; `end` = shuttle down / fault; a shot = any racket?shuttle
contact by either player on a side.
133 - `shots_count` = total contacts across both teams in the rally. Do not track
which player hit ? just count every contact.
134 - Net exchanges (drives, flat pushes) come fast ? frame-step so you neither miss nor
double-count contacts.
135 - Only the designated receiver returns the serve; if the wrong player returns, the
rally ends on a fault ? still a row (`service_fault` if it's a serve/receiving-position fault,
else `other` + note).
136 - Two teammates may not both hit the shuttle in succession (double-hit). If you see
two contacts on the same side back-to-back, that's a fault ending the rally ? count both
contacts, then the rally ends.
137
138 ### Pickleball
139 - Serve: underhand, paddle below waist, hit diagonally; must clear the non-volley
zone (the "kitchen"). Start = paddle?ball contact.
140 - Two-bounce rule: the serve must bounce once in the receiver's court and the return
must bounce too, before either side may volley. These bounces are normal play, not rally
ends, and bounces are not shots.
141 - Rally end: ball bounces twice, lands out, hits the net (fails to cross), or a
fault (e.g. volleying in the kitchen).
142 - Shots: every paddle?ball contact only; serve = shot 1; floor bounces are not
shots.
143 - Service faults (`service_fault`): serve into net, out/long, short in the kitchen,
foot fault.
144
145 Worked example:
146 ```
147 [SCREENSHOT: pickleball serve + kitchen line]
148 Rally 2: serve 0:33.10; serve bounces, return bounces, a few volleys; receiver nets an
easy one at 0:41.85; 6 paddle contacts.
149 2, 0:33.10, 0:41.85, (auto), (auto), 6, into_net, 1-0, 2-0,
150 ```
151
152 Pickleball ? DOUBLES (the standard format, 2 per side). All rally rules
identical to singles:
153 - `start` = serve contact; a shot = any paddle?ball contact by either teammate;
floor bounces still not shots; two-bounce rule still applies.
154 - `shots_count` = total paddle contacts across both teams; don't track which player.

```

```

155 - **Serving sequence (FYI only ? you do NOT annotate it):** both partners serve before
the serve passes to the opponents, except the very first service turn of the game. The called
score has three numbers (server score ? receiver score ? server #1/#2). Ignore this for
annotation; just mark boundaries / shots / ending.
156 - If both teammates contact the ball in succession, that's a fault ? rally ends; count
the contacts, set the ending per the fault.
157
158 ### Table tennis
159 - **Serve:** ball tossed up from an open palm, then struck so it bounces once on each
side. Start = the **racket?ball strike** (not the toss).
160 - **Rally end:** a player fails a legal return ? ball double-bounces on their side, goes
off the table, or is hit into the net on a non-serve shot.
161 - **Shots:** every racket?ball contact only; serve = shot 1; table bounces are not
shots.
162 - **Let:** a serve that clips the net but is otherwise legal ? the point is replayed.
`ending_reason` = let`, and the replay is its own next row.
163 - **Service faults** (`service_fault`): missed/illegal toss, serve into net, serve off
the table.
164 - **Speed warning:** contacts can be < 0.2 s apart ? **step frame-by-frame** to count
shots.
165 - **Doubles (if a clip appears):** partners must hit in **strict alternation** and serve
diagonally; a missed alternation is a fault that ends the rally. Same rule ? count **every**
racket?ball contact as a shot; `shots_count` is the team total.
166
167 Worked example:
168 ```
169 [SCREENSHOT: table-tennis serve contact]
170 Rally 9: serve 4:10.40; fast 7-contact rally; forehand winner, opponent no contact; dead
4:14.05.
171 9, 4:10.40, 4:14.05, (auto), (auto), 7, winner, 6-9, 6-10,
172 Rally 10: serve 4:22.10 clips the net (otherwise good) -> let.
173 10, 4:22.10, 4:23.00, (auto), (auto), 1, let, 6-10, 6-10, net-cord serve, replayed
174 ```
175
176 ## Part 6 ? Edge cases & FAQ
177 - **Two rallies with almost no gap?** Still two rows. `end_time[N] <= start_time[N+1]`,
never overlap.
178 - **Interrupted & replayed?** Label the interrupted attempt as its own row
(`ending_reason=let`), then the replayed point as the next row.
179 - **Serve tossed but caught / re-served (no contact)?** No contact = no rally yet. Start
at the actual serve contact; don't label the aborted toss.
180 - **Opponent reached the ball but netted/hit out?** That's `into_net`/`out` (their
shot), not `winner`.
181 - **Can't read the scoreboard?** Leave `score_before`/`score_after` blank. Never guess.
182 - **"Let" where the rules say play on?** If play continued, it's **not** a let ? keep
one rally and judge the real ending.
183 - **Warm-up / knock-up / practice points?** Not real rallies ? **do not label them.**
Start at the first real, scored serve. See the **Rule 2 caveat (Part 2)** for how to tell warm-
up from real play (no serve, cooperative not competitive, score not progressing, several
shuttles/balls in use). When unsure whether the match has started, flag the clip ? don't guess.
184 - **Doubles?** Same rules ? a shot is any one racket/paddle contact by whichever player
hits it. See the badminton and pickleball **DOUBLES** notes in Part 5; never attribute
`shots_count` to an individual, just total the contacts.
185 - **Server completely whiffs (no contact)?** Rare. True zero contact ? `shots_count` =
0`, `ending_reason` = service_fault`, add a note. A graze = 1 contact = 1 shot.
186
187 ## Part 7 ? Self-check before you submit (per video)
188 - [ ] `rally_number` is 1,2,3? no gaps or repeats.
189 - [ ] Every `start_time < end_time` (check the auto-filled decimal columns populated).
190 - [ ] No overlaps: `end_time[N] <= start_time[N+1]`.
191 - [ ] Times entered as `m:ss.mmm`; decimal `start_time`/`end_time` auto-filled and look
right.
192 - [ ] No dead-time rows (scan the gaps ? dead time, not missing rallies).
193 - [ ] Every served point present, including service faults and lets.
194 - [ ] `shots_count` looks right (re-step any rally you rushed; doubles = team total).

```



```

195     - [ ] `ending_reason` ? {`winner`,`into_net`,`out`,`service_fault`,`let`,`other`}; notes
filled when `other`.
196     - [ ] File downloaded as `**not** trim / re-encode / rename the source video.
198
199     ## Part 8 ? Mini glossary
200     - **Rally** ? one continuous passage of play from serve contact to the shuttle/ball
dead.
201     - **Shot** ? one contact between the shuttle/ball and a racket/paddle/bat (either
teammate in doubles).
202     - **Dead time** ? any time the shuttle/ball is not in active play. Never labeled.
203     - **Service fault** ? a failed/illegal serve that ends the point. Still a labeled rally.
204     - **Let** ? a replayed point. Labeled as its own rally with `ending_reason=let`.
205     - **Winner / into_net / out** ? the final shot landed in untouched / went into the net /
landed out.
206     - **start_time / end_time** ? decimal seconds from the start of the provided file (auto-
filled from mm:ss).
207

```

7. user (2026-06-06T06:45:19.621Z)

```

1     video_id,sport,format,match_name,source_filename,drive_link,fps,duration_mmss,expected_r
allies,assigned_to,status,notes
2     pilot_badminton_01,badminton,singles,TODO(avidullu): match name,TODO(avidullu): exact
source filename,TODO(avidullu): paste Drive link to this video,TODO(avidullu): fps e.g.
30,TODO(avidullu): length e.g. 2:45,10-12,,todo,deliver as pilot_badminton_01.csv
3     pilot_pickleball_01,pickleball,doubles,TODO(avidullu): match name,TODO(avidullu): exact
source filename,TODO(avidullu): paste Drive link to this video,TODO(avidullu): fps e.g.
30,TODO(avidullu): length e.g. 2:30,10-12,,todo,deliver as pilot_pickleball_01.csv
4     pilot_tabletennis_01,tabletennis,singles,TODO(avidullu): match name,TODO(avidullu):
exact source filename,TODO(avidullu): paste Drive link to this video,TODO(avidullu): fps e.g.
30,TODO(avidullu): length e.g. 2:20,10-12,,todo,deliver as pilot_tabletennis_01.csv
5

```

8. assistant (2026-06-06T06:45:25.100Z)

I have everything I need from both spec PDFs and the collector source-of-truth docs. Now let me verify a couple of the established facts directly against the delivered files before finalizing, particularly the ending_reason vocabulary actually delivered and the time-computation hypothesis.

9. user (2026-06-06T06:45:29.636Z)

```

JOB id 4075945
JOB size 710
JOB mode interpolation
JOB overlap 0
JOB bugtracker None
JOB created 2026-06-04 06:07:18.873191+00:00
JOB updated 2026-06-05 09:47:11.609271+00:00
JOB subset Test
JOB start_frame 0
JOB stop_frame 21270
JOB frame_filter step=30
JOB segments

```

JOB owner

JOB assignee None
JOB labels

```

LABELS ['rally']
ATTR NAMES ['rally_number', 'start_time', 'end_time', 'shots_count', 'ending_reason',
'score_before (A-B)', 'score_after (A-B)', 'notes']

```

10. user (2026-06-06T06:45:34.825Z)

ENDING_REASON COUNTS {}
NUM DISTINCT RALLIES 0

11. assistant (2026-06-06T06:45:39.122Z)

The attributes are stored differently than I parsed (likely the `attribute` elements use a different structure, or attributes are inline). Let me inspect the raw XML structure of a box element.

12. user (2026-06-06T06:45:41.937Z)

Exit code 1
Traceback (most recent call last):
 File "<string>", line 5, in <module>
 tr = r.findall('.//track')[0]
 ~~~~~^  
IndexError: list index out of range

13. assistant (2026-06-06T06:45:46.942Z)

No tracks. This export uses `<image>` elements (PASCAL/image-based) rather than `<track>`. Let me check the top-level structure.

14. user (2026-06-06T06:45:50.322Z)

ROOT annotations  
TOP children tags: Counter({'image': 710, 'version': 1, 'meta': 1})  
num image elements: 710  
IMAGE attrib: {'id': '150', 'name': 'frame\_000150', 'width': '1920', 'height': '1080'}  
BOX attrib: {'label': 'rally', 'source': 'manual', 'occluded': '0', 'xtl': '1134.10', 'ytl': '434.73', 'xbr': '1286.29', 'ybr': '705.46', 'z\_order': '0'}  
 attr {'name': 'rally\_number'} '2'  
 attr {'name': 'start\_time'} '2'  
 attr {'name': 'end\_time'} '2.1'  
 attr {'name': 'shots\_count'} '2'  
 attr {'name': 'ending\_reason'} 'forced\_error'  
 attr {'name': 'score\_before (A-B)'} None  
 attr {'name': 'score\_after (A-B)'} None  
 attr {'name': 'notes'} None

15. assistant (2026-06-06T06:45:53.332Z)

Now I can parse correctly. The attribute element tag is `attribute`, and the value is the text. Let me collect all ending\_reason values and verify the time hypothesis across rallies.

16. user (2026-06-06T06:45:59.920Z)

ENDING\_REASON COUNTS: {'forced\_error': 45, 'other': 92, 'unforced\_error': 41, 'winner': 131, 'service\_fault': 9}  
NUM DISTINCT rally\_number: 24

| rally | minframe | maxframe | start_time | end_time | minf/900 | maxf/900 |
|-------|----------|----------|------------|----------|----------|----------|
| 1     | 180      | 1260     | st= 0.2    | et= 1.4  | 0.2      | 1.4      |
| 2     | 150      | 1950     | st= 2      | et= 2.1  | 0.17     | 2.17     |
| 3     | 2670     | 3060     | st= 2.9    | et= 3.4  | 2.97     | 3.4      |
| 4     | 3720     | 4020     | st= 4.13   | et= 4.14 | 4.13     | 4.47     |
| 5     | 4320     | 4560     | st= 4.8    | et= 5.06 | 4.8      | 5.07     |
| 6     | 4980     | 5340     | st= 5.53   | et= 5.9  | 5.53     | 5.93     |
| 7     | 5730     | 6690     | st= 6.36   | et= 6.66 | 6.37     | 7.43     |
| 8     | 7080     | 7170     | st= 7.86   | et= 7.96 | 7.87     | 7.97     |
| 9     | 7590     | 7860     | st= 8.43   | et= 8.73 | 8.43     | 8.73     |
| 10    | 8280     | 8460     | st= 9.2    | et= 9.4  | 9.2      | 9.4      |

|    |       |       |                     |       |       |
|----|-------|-------|---------------------|-------|-------|
| 11 | 8970  | 9240  | st= 9.96 et= 10.23  | 9.97  | 10.27 |
| 12 | 9780  | 9900  | st= 10.86 et= 10.96 | 10.87 | 11.0  |
| 13 | 10410 | 10680 | st= 11.56 et= 11.86 | 11.57 | 11.87 |
| 14 | 10980 | 11160 | st= 12.2 et= 12.4   | 12.2  | 12.4  |
| 15 | 11550 | 11760 | st= 12.83 et= 13.06 | 12.83 | 13.07 |
| 16 | 12330 | 13380 | st= 13.7 et= 13.93  | 13.7  | 14.87 |
| 17 | 14160 | 14250 | st= 15.73 et= 15.73 | 15.73 | 15.83 |
| 18 | 14580 | 15120 | st= 16.2 et= 16.8   | 16.2  | 16.8  |
| 19 | 15750 | 15900 | st= 17.5 et= 17.66  | 17.5  | 17.67 |
| 20 | 16230 | 16830 | st= 18.03 et= 18.7  | 18.03 | 18.7  |
| 21 | 17670 | 17820 | st= 19.63 et= 19.8  | 19.63 | 19.8  |
| 22 | 18480 | 18720 | st= 20.53 et= 20.8  | 20.53 | 20.8  |
| 23 | 19260 | 19500 | st= 21.4 et= 21.66  | 21.4  | 21.67 |
| 24 | 19920 | 20160 | st= 22.13 et= 22.4  | 22.13 | 22.4  |

#### 17. assistant (2026-06-06T06:46:08.136Z)

This confirms the key facts decisively. The `start\_time` attribute equals `min\_frame / 900` and `end\_time` equals approximately `max\_frame / 900` (specifically the last \*sampled\* frame of the rally / 900). The video is 1080p at standard fps, but the times are being divided by 900 instead of the true fps (~30), so all timestamps are compressed by a factor of ~30. The frame ranges are the real temporal signal.

Note the `ending\_reason` vocabulary delivered: `winner, forced\_error, unforced\_error, service\_fault, other` (and the KNOWN\_FACTS mentioned `let` too ? let me confirm whether `let` actually appears, since my counts didn't show it).

The delivered ending\_reason values are: `forced\_error, other, unforced\_error, winner, service\_fault`. No `let` and no `into\_net`/`out` appeared in this badminton job. Let me also quickly confirm the COCO/VOC folders are the same badminton job (no pickleball/TT) and check exact field naming in the COCO export.

#### 18. user (2026-06-06T06:46:11.884Z)

```
TOP-LEVEL FOLDERS: ['badminton', 'badminton (1)', 'badminton (2)']
COCO keys: ['licenses', 'info', 'categories', 'images', 'annotations']
COCO #images: 710 #annotations: 318 #categories: 1
COCO categories: [(1, 'rally')]
COCO sample image filenames: ['frame_000000.png', 'frame_000030.png', 'frame_000060.png']
COCO ann0 bbox: [1134.1, 434.73, 152.19, 270.73] attributes: {'rally_number': 2.0, 'start_time': '2', 'end_time': '2.1', 'shots_count': 2.0, 'ending_reason': 'forced_error', 'score_before (A-B)': '', 'score_after (A-B)': '', 'notes': '', 'occluded': False, 'rotation': 0.0}
```

#### 19. assistant (2026-06-06T06:46:54.973Z)

All facts are now confirmed. The COCO bbox `[1134.1, 434.73, 152.19, 270.73]` matches the VOC/CVAT box for rally 2 (x1=1134.10, y1=434.73; width 152.19 = 1286.29-1134.10; height 270.73 = 705.46-434.73). All three folders are the same badminton job in three formats. Only `badminton` is delivered ? no pickleball, no table tennis.

I have everything needed. Let me compile the final structured answer.

#### 20. user (2026-06-06T06:47:18.174Z)

Structured output provided successfully

#### 21. assistant (2026-06-06T06:47:33.225Z)

I've completed the analysis. Summary of what I established:

**\*\*What was requested (grounded in the PDFs actually sent to Roora):\*\***

- Format: one per-rally CSV per video (`<video\_id>.csv`), one row per rally, decimal-second timestamps. CVAT/Label Studio explicitly allowed only as a working tool ? "the final DELIVERABLE must be the CSV." The PDF even says it "supersedes the earlier CVAT-centric brief."

- 8 fields: rally\_number, start\_time, end\_time, shots\_count, ending\_reason, score\_before, score\_after, notes.
- ending\_reason vocab (PDF): `winner | forced\_error | unforced\_error | service\_fault | let | other`.
- Scope: 3 pilot videos ? 1 badminton + 1 pickleball + 1 table tennis, each <3 min, ~10-12 rallies.

**\*\*Deviations verified against the delivery (blockers):\*\***

1. Wrong format ? bbox export (CVAT XML + COCO JSON + PASCAL VOC, same job 3x) instead of a CSV.
2. Scope ? only badminton; pickleball and table tennis entirely missing.
3. Broken times ? confirmed `start\_time == frame/900`, `end\_time == last-sampled-frame/900` across all 24 rallies (e.g. rally 24: 19920/900=22.13, 20160/900=22.4). 900 ? fps, so all timestamps are ~30x compressed; the real signal is the frame range.
4. Source mismatch ? annotated source is 21270 frames @1920x1080; the on-disk Video 1 is 3019 frames @5312x2988. Different file, violating the no-re-encode/checksum rule.

**\*\*Corrections to the KNOWN\_FACTS I was given:\*\***

- `let` was NOT actually used ? values present are winner/forced\_error/unforced\_error/service\_fault/other. So the ending\_reason vocab is compliant (no out-of-vocab values).
- All `notes` are blank even where `ending\_reason=other` (~29% of boxes), violating "notes REQUIRED when other."
- 24 rallies, ~2x the expected 10-12 ? another signal the wrong/longer source was annotated.

**\*\*One doc-hygiene finding worth flagging:\*\*** our own `docs/annotation/ROORA\_BRIEF.md` (markdown) uses a *different* ending\_reason vocab (`into\_net|out` instead of `forced\_error|unforced\_error`) and extra `start\_mmss/end\_mmss` columns than the PDF actually sent. The PDFs are the contract, so Roora's vocab choice was correct ? but you may want to reconcile the two docs before Phase B. Relevant paths: `C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ROORA\_BRIEF.md` vs the two PDFs under `C:\Users\avidu\Projects\Annotation Setup\`.